

Khwopa College of Engineering
OOP Lab Sheet
I Year/ I Part
Faculty: Computer and Electronics
Labsheet #1: Basics of Programming with C++

Objectives:

1. Working of cin , cout , insertion operator (<<) and extraction operator (>>) .
[Note: The extraction operator(>>) is used along with the object cin for reading inputs. The insertion operator(<<) is used along with the object cout for displaying data on the screen.]
2. Call by Value and Call by Reference.
3. Structure in C++.

Tasks:

Objective#1:

Show the input as well as output to your teacher for the following program.

```
#include <iostream>
using namespace std;
int main()
{
    int height;
    cout << "Enter your height:\n";
    cin >> height;
    cout << "My height is: " << height;
    return 0;
}
```

Assignment:

1. WAP to add and multiply two numbers.
2. WAP to find out greatest number among three numbers.
3. WAP to find Fibonacci numbers (0,1,1,2,3,5,.....).

Objective#2:

1. Call by Value:

```
#include <iostream>
using namespace std;
void call_by_val (int a) {
    a=20;
    cout<<"Value of a from this function: "<< a << endl;
}
int main()
{
    int x = 100;
    call_by_val (x);
    cout<<"Value of a from main function: "<< x ;
}
```

```

    return 0;
}

```

Assignment:

1. Show the input as well as output of above program to your teacher.
2. WAP to swap two numbers using the concept of call by value.

2. Call by Reference:

```

#include <iostream>
using namespace std;
void call_by_ref (int &a) {
    a=20;
    cout<<"Value of a from this function: "<< a << endl;
}
int main()
{
    int x = 100;
    call_by_ref(x);
    cout<<"Value of a from main function: "<< x ;
    return 0;
}

```

Assignment:

1. Show the input as well as output to your teacher.BCT/BEL,
2. WAP to swap two numbers using the concept of call by reference.

Objective #3

```

#include <iostream>
using namespace std;
struct Student
{
    int roll_no;
    int age;
};
int main()
{
    struct Student S;
    S.roll_no=2;
    S.age=18;
    cout<<"Her Roll Number is : "<< S.roll_no << endl;
    cout<<"Her age is : "<< S.age;
    return 0;
}

```

Assignment:

1. Show the input as well as output to your teacher.
2. Write a program to store the roll no. (starting from 1), name and age of 5 students and then print the details of the student with roll no. 4. Write a program to add two distances in inch-feet using structure. The values of the distances is to be taken from the user.

Khwopa College of Engineering

OOP Lab Sheet

I Year/ I Part

Faculty: Computer and Electronics

Labsheet #2: Functions in C++

Objectives:

1. Function Overloading in C++
2. Default Argument
3. Inline Function

Tasks:

Objective#1: Function Overloading in C++

When a function name is overloaded with different jobs, it is called Function Overloading.

It is a feature of object oriented programming (OOP) where two or more functions can have the same name but have different parameter list (when we say parameter list, it means the data type and sequence of the parameters). E.g:

- `int add (int num1, float num2) { }`
- `int add (int num1, int num2, int num3) { }`
- `int add (int num1, int num2) { }`
- `int add (float num1 , int num2) { }`

The easiest way to remember this rule is that the parameters should qualify any one or more than one of the following conditions:

- They should have a different type
- They should have a different number
- They should have a different sequence of parameters

Show the input as well as output to your teacher for the following program and discuss the result.

```
#include <iostream>
using namespace std;
// function with two parameter
void add(int num1, int num2) {
    cout<< "Hey!! You are calling function with two parameter of
        type int, int..... "<<endl;
    cout << "Sum of num1 and num2 is: " << num1+num2 <<
        endl<<endl;
}
// function with three parameters
void add(int num1,int num2, float num3) {
    cout<< "Hey!!You are calling function with three parameter of
        type int, int ,int..... "<<endl;
    cout << "Sum of num1, num2 and num3 is: " <<
        num1+num2+num3<<endl<<endl;
}
// function with two parameter
```

```

void add(int num1, float num2) {
    cout<< "Hey!!You are calling function with two parameter of
           type int, float..... "<<endl;
    cout << "Sum of num1 and num2 is: " << num1+num2 <<
           endl<<endl;
}
int main() {
    int a=10, b=20;
    float c=10.10;
    add(a,b);
    add(a,b,c);
    add(a,c);
    return 0;
}

```

Assignment:

1. WAP to calculate and display the cube of integer, float and double number using function overloading.

Objective#2: Default Argument:

A default argument is a value provided in a function declaration that is automatically assigned by the compiler if the caller of the function doesn't provide a value for the argument with a default value. In case any value is passed then the default value is overridden.

#Example 1

```

#include <iostream>
using namespace std;
int add(int a, int b, int c= 10, int d = 20)
{
    return (a + b + c + d);
}
int main()
{
    cout << add(30, 20) << endl;
    cout << add(20, 30, 40) << endl;
    cout << add(20, 10, 30, 40) << endl;
    return 0;
}

```

#Example 2

```

#include <iostream>
using namespace std;
// Program to demonstrate Function overloading in Default
Arguments.....
int add(int a, int b, int c = 10, int d= 20)
{
    cout<<"Calling first function"<<endl;
}

```

```

    return (a + b + c + d);
}
double add(int a, double b, double c = 10.1, double d= 20.1)
{
    cout<<"Calling second function"<<endl;
    return (a + b + c + d);
}
int main()
{
    cout << add(10, 15) << endl;
    cout << add(10, 15.1, 25.1) << endl;
    cout << add(10, 15, 25, 30) << endl;
    return 0;
}

```

Assignment:

1. Show the input as well as output of above program to your teacher and discuss the output.
2. WAP to find the area of triangle (where three sides are given) and area of rectangle using function overloading and default argument..

Objective #3: Inline Function

C++ provides an inline functions to reduce the function call overhead. Inline function is a function that is expanded in-line when it is called. When the inline function is called, whole code of the inline function gets inserted or substituted at the point of inline function call. This substitution is performed by the C++ compiler at compile time. Inline function may increase efficiency if it is small.

****Remember,** inlining is only a request to the compiler, not a command. Compiler can ignore the request for inlining.

Compiler may not perform inlining in circumstances like:

1. If a function contains a loop. (for, while, do-while)
2. If a function contains static variables.
3. If a function is recursive.
4. If a function return type is other than void, and the return statement doesn't exist in function body.
5. If a function contains switch or goto statement.

Syntax :

```

inline return-type function-name(parameters)
{
    // function body
}

```

Example:

```
#include <iostream>
#include<math.h>
using namespace std;
inline int power(int a) //inline function
{
    return pow(a,2);
}
int main()
{
    cout << "The power of 2 is: " << power(2);
    return 0;
}
```

Assignment:

1. Check execution time of a program using inline keyword and without using inline keyword and discuss the result.

Khwopa College of Engineering
OOP Lab Sheet
I Year/ I Part
Faculty: Computer and Electronics
Labsheet #3:

Objectives:

1. Class and Object in C++
2. Constructor and Destructor

Tasks:

Objective#1: Class and Object in C++

Class :

A Class in C++ is the building block that leads to OOP. A C++ class is like a blueprint for an object. A Class is a user defined data-type which has its data members and member functions, which can be accessed and used by creating an instance of that class. Data members are the data variables and member functions are the functions used to manipulate these variables and together these data members and member functions defines the properties and behavior of the objects in a Class.

E.g. If we take Student as class then name, age, roll_no. etc are its data members and eat() , sleep() etc are its member functions.

Syntax:

```
class class_name {           //here, class is keyword
    Access_specifier:        // can be private ,public and protected
    Data members;            //variables to be used
    Member functions() {}    //methods or functions to access data members
};                            //class should end with semi colon.
```

Object:

An Object is an instance of a Class. When a class is defined, no memory is allocated but when it is instantiated (i.e. an object is created) memory is allocated.

Syntax:

```
Class_name Object_name;
```

[**Accessing data members and member functions of class: The data members and member functions of class can be accessed using the dot('.') operator with an object.]

Show the input as well as output to your teacher for the following program and discuss the result.

```
#include <iostream>
using namespace std;
class Student {
    private:
        int roll_no;
        string name;
        int age;
    public:
        void input() {
            cout<<"Enter your Roll_no: ";
            cin>>roll_no;
            cout<<"Enter your name: ";
            cin>>name;
            cout<<"Enter your age: ";
            cin>>age;
        }
        void display() {
            cout<<"Full Detail of Student: "<<endl;
            cout<<"Roll No.: "<<roll_no<<"\nName: "
                <<name<<"\nAge: "<<age;
        }
};
int main() {
    Student s;
    s.input();
    s.display();
    return 0;
}
```

Assignment:

1. WAP that illustrates OOP concept to add and subtract time units (a time unit has hr, min and sec as data member).
2. Write a temperature conversion program that gives the user the option of converting degree Fahrenheit to degree Celsius or vice versa using the concept of class and object.
3. WAP print the sum, difference and product of two complex numbers by creating a class named 'Complex' with separate functions for each operation whose real and imaginary parts are entered by the user.
4. WAP that would print the information (Name, Address ,Joining_Year, Salary) of three employees by creating a class named 'Employee'.

Name	Joining_Year	Address	Salary
Manoram	2000	Kathmandu	50000
Ram	2002	Bhaktapur	40000
Sita	2005	Lalitpur	55000

Objective#2: Constructor and Destructor:

Constructor in C++ is a special member function of a class whose task is to initialize the object of the class.

Syntax:

```
Class_name () { //Constructor's body }
```

A destructor is also a member function of a class that is instantaneously called whenever an object is destroyed.

Syntax:

```
~Class_name () { //Destructor's body }
```

Basically there are three types of constructors namely : Default Constructor, Parameterized Constructor and Copy Constructor.

#Example

```
class ABC
{
    public:
        ABC() // constructor .This is Default Constructor.
        {
            cout<<"Constructor called"<<endl;
        }
        ~ABC() // destructor
        {
            cout<<"Destructor called"<<endl;
        }
};

int main()
{
    ABC A; // Constructor Called
    int x = 10;
    if(x==10)
    {
        ABC A1; // Constructor Called
    } // Destructor Called for A1
} // Destructor called for A
```

Assignment:

1. WAP to display students details using constructor and destructor.
2. WAP print the sum, difference and product of two complex numbers by creating a class named 'Complex' with separate functions for each operation whose real and imaginary parts are entered by the user. (Using the concept of Constructor and Destructor.)
3. WAP with class named Book, which will represent title, author, price , publisher. Use constructor for initialization of an objects and use dynamic memory allocation for string type member and destructor in a proper way.

Khwopa College of Engineering
OOP Lab Sheet
I Year/ I Part
Faculty: Computer and Electronics
Labsheet #4:

Objectives:

1. Static Member Variable and Static Member Function
2. Friend Function and Friend Class

Tasks:

Objective#1: Static member Variable and Static Member Function:

Static member Variable:

- Declared inside the class body and also known as class member variable.
- They must be defined outside the class.
- They does not belong to any object but to the whole class.
- There will be only one copy of static member variable for the whole class.
- Any object can use the same copy of class variable.
- They can also be used with class name.

Example:

```
<#include <iostream>
using namespace std;
class Sample{
    public:
        int x;    //instance member variable static
        int y;    //static member variable or class member
                variable
};
int Sample::y=10; //static member variable defined outside the
class.
int main() {
    cout<<Sample::y<<endl;           //accessing without object
    Sample s1;
    cout<<s1.y;                       //accessing with object
    return 0;
}
```

Static Member Function:

- They are qualified with the keyword **static**.
- They are also called class member function.
- They can be invoked with or without object.
- They can only access static member of the class.

Example:

```
#include <iostream>
using namespace std;
class Sample {
    private:
        int x;
        static int y;
    public:
        static void display(){ //static member function or
                                class member function
                                cout<<y<<endl;
        }
};
int Sample::y=10;
int main()
{
    Sample::display();          //accessing without object
    Sample s1;
    s1.display();               //accessing with object
    return 0;
}
```

Assignment:

1. WAP to illustrate the use of static data variable and static member function to show the order of invocation of constructor and destructor.

Objective#2: Friend Function and Friend Class:

Friend Function:

- It is not a member function of a class to which it is friend.
- It is declared in the class with friend keyword.
- It must be defined outside the class to which it is friend.
- It can access any member of the class to which it is friend.
- A friend function of a class is defined outside that class scope but it has the right to access all private and protected members of the class.
- It can not access members of the class directly.
- It has no caller object.
- It should not be defined with membership label.
- Friend function can become friend to more than one class.(it is unique behavior of friend function).

#Example

```
#include <iostream>
using namespace std;
class Sample{
    int x;
    public:
        Sample() {
            x=0;
        }
}
```

```

        void display(){
            cout<<"Value of x is: "<<x<<endl;
        }
        friend void add(Sample &s); //friend function's prototype.
            //pass by reference is opt.
        };
        void add(Sample &s){ //friend function definition.
            s.x=s.x+4;
        }
int main()
{
    Sample s1;
    s1.display();
    add(s1); //friend function call.
    s1.display();
    return 0;
}

```

Assignment:

1. WAP to find sum of two distance (feet, inch) objects using friend function.

Friend Class:

- A friend class can access private and protected members of other class in which it is declared as friend. It is sometimes useful to allow a particular class to access private members of other class.
- Following are some important points about friend functions and classes:
 1. Friends should be used only for limited purpose.
 2. Friendship is not mutual. If class **A** is a friend of **B**, then **B** doesn't become a friend of **A** automatically.
 3. Friendship is not inherited.
 4. The concept of friends is not there in Java

#Example

```

#include <iostream>
using namespace std;
class A {
int x;
public:
    A(){
        x=0;
    }
    friend class B;
};
class B{
public:
    void display(A &a){ //pass by reference is opt.
        cout<<"Value of x: "<<a.x<<endl;
    }
};
int main() {

```

```
A  a1;  
B  b1;  
b1.display(a1);  
return 0;  
}
```

Assignment:

1. Create a class mdistance to store the values in meter and centimeter and class edistance to store values in feet and inches. Perform addition of object of mdistance and object of edistance by using friend function.

Khwopa College of Engineering
OOP Lab Sheet
I Year/ I Part
Faculty: Computer and Electronics
Labsheet #5

Objectives:

1. Binary Operator Overloading and Unary Operator Overloading
2. Data Conversion: Basic to User-Defined Type, User-Defined Type to Basic Type, User-Defined to User-Defined Type

Tasks:

1. WAP to concatenate two user given string using the concept of Operator Overloading.
2. WAP to overload - - (pre decrement and post decrement both) operator.
3. WAP to create class named LandMeasure that have Ropani, Ana , Paisa and Dam as Data Members. Write a function to pass two object of type LandMeasure and return their sum using the Concept of Operator Overloading.
4. WAP that will add objects of a Date class with member variables day ,month and year using Operator Overloading.
5. WAP to convert currency from ruppee to dollar using both concept i.e. using constructor for data conversion as well as casting operator for data conversion.
6. WAP that converts object of Celsius type to object of Fahrenheit type.
7. WAP that will convert object from a class Rectangle to object of a class Polar using Casting Operator.

Binary Operator Overloading:

Operator + (e.g. S1+S2):

OO with Member Function:

```
#include <iostream>
using namespace std;

class Sample{
    int x ,y;
public:
    Sample():x(0),y(0){}
    Sample(int a, int b):x(a),y(b){}
    Sample operator +(Sample s){
        Sample temp;
        temp .x =x +s.x;
        temp .y=y+ s.y;
        return temp;
    }
    void display(){
        cout<<"Value of x: "<<x<<" and y: "<<y<<endl;
    }
};
```

```

int main(){
    Sample s1(2,3) ,s2(4,5) ,s3;
    s3=s1+s2; //s3=s1.operator+(s2)-->both are equivalent.
    s3.display();
    return 0;
}

```

OO with Friend Function:

```

#include <iostream>
using namespace std;
class Sample{
    int x, y;
public:
    Sample():x(0),y(0) {}
    Sample(int a, int b):x(a),y(b){}

    void display(){
        cout<<"Value of x: "<<x<<" and y: "<<y<<endl;
    }
    friend Sample operator +(Sample, Sample);
};
Sample operator +(Sample s11,Sample s22){
    Sample temp;
    temp.x =s11.x+s22.x;
    temp.y =s11.y+s22.y;
    return temp;
}
int main(){
    Sample s1(2,3), s2(4,5), s3;
    s3=s1+s2;
    s3.display();
    return 0;
}

```

Unary Operator Overloading:

Operator Negation(-) (e.g. -S):

OO with Member Function:

```

#include <iostream>
using namespace std;
class Sample{
    int x, y;
public:
    Sample():x(0),y(0){}
    Sample(int a ,int b):x(a),y(b){}
    Sample operator -(){
        Sample temp;
        temp.x=-x;
        temp.y=-y;
        return temp;
    }
}

```

```

        void display(){
            cout<<"Value of x: "<<x<<" and y: "<<y<<endl;
        }

};

int main(){
    Sample s1(2,3),s2;
    s2=-s1;//s2=s1.operator -();--> both are equivalent..
    s2.display();
    return 0;
}

```

OO with Friend Function:

```

#include <iostream>
using namespace std;
class Sample{
    int x, y;
public:
    Sample():x(0),y(0){}
    Sample(int a ,int b):x(a),y(b){}

    void display(){
        cout<<"Value of x: "<<x<<" and y: "<<y<<endl;
    }
    friend Sample operator -(Sample);
};

Sample operator -(Sample s){
    Sample temp;
    temp.x=-s.x;
    temp.y=-s.y;
    return temp;
}

int main(){
    Sample s1(2,3),s2;
    s2=-s1;
    s2.display();
    return 0;
}

```

Pre-increment (e.g. ++S):

OO with Member Function:

```
#include <iostream>
using namespace std;
class Sample{
    int x, y;
public:
    Sample():x(0),y(0){}
    Sample(int a, int b):x(a),y(b){}
    Sample operator ++(){
        Sample temp;
        temp.x=++x;
        temp.y=++y;
        return temp;
    }
    void display(){
        cout<<"Value of x: "<<x<<" and y: "<<y<<endl;
    }
};
int main(){
    Sample s1(2,3),s2;
    s1.display();
    s2=++s1;
    s2.display();
    return 0;
}
```

OO with Friend Function:

```
#include <iostream>
using namespace std;
class Sample{
    int x, y;
public:
    Sample():x(0),y(0){}
    Sample(int a ,int b):x(a),y(b){}

    void display(){
        cout<<"Value of x: "<<x<<" and y: "<<y<<endl;
    }
    friend Sample operator ++(Sample);
};
Sample operator ++(Sample s){
    Sample temp;
    temp.x=++s.x;
    temp.y=++s.y;
    return temp;
}
```

```

int main(){
    Sample s1(2,3),s2;
    s1.display();
    s2=++s1;
    s2.display();
    return 0;
}

```

Post-increment (e.g. S++):

OO with Member Function:

```

#include <iostream>
using namespace std;
class Sample{
    int x,y;
public:
    Sample():x(0),y(0){}
    Sample(int a, int b):x(a),y(b){}
    Sample operator ++(int){//Here,int is just a dummy..
        Sample temp;
        temp.x=x++;
        temp.y=y++;
        return temp;
    }

    void display(){
        cout<<"Value of x: "<<x<<" and y: "<<y<<endl;
    }
};

int main(){
    Sample s1(2,3),s2;
    s1.display();
    s2=s1++;
    s2.display();
    s1.display();
    return 0;
}

```

OO with Friend Function:

```

#include <iostream>
using namespace std;
class Sample{
    int x, y;
public:
    Sample():x(0),y(0){}
    Sample(int a, int b):x(a),y(b){}
    void display(){
        cout<<"Value of x: "<<x<<" and y: "<<y<<endl;
    }
    friend Sample operator ++(Sample &,int);
};

```

```

Sample operator ++(Sample &s ,int){
    Sample temp;
    temp.x=s.x++;
    temp.y=s.y++;
    return temp;
}
int main() {
    Sample s1(2,3),s2;
    s1.display();
    s2=s1++;
    s2.display();
    s1.display();
    return 0;
}

```

Data Conversion:

1. Basic to User-Defined Type:

//1.Basic/Primitive/In-built Type to User-Defined/Class Type..
 //--> Primitive type to Class/User-Defined type can be implemented
 // through Constructor.

```

#include <iostream>
using namespace std;

class Sample{
    int x ,y;
public:
    Sample(){x=0;y=0;}      //Default Constructor

    Sample(int m){
        x=y=m;
    }
    void display(){
        cout<<"Value of x is: "<<x<<" and y is: "<<y<<endl;
    }
};

int main(){
    int a=10;
    Sample s1;
    s1=a;    //Here, a is primitive type and s1 is class type.
    s1.display();
    return 0;
}

```

2. User-Defined to Basic Type:

```
//2. Class /User-Defined Type to Basic/Primitive Type..
//--> Class/User-Defined Type to Basic/Primitive Type can be
//      implemented through casting operator.
#include <iostream>
using namespace std;
class Sample{
    int x ,y;
public:
    Sample(int a, int b):x(a),y(b){}
    void display(){
        cout<<"Value of x is: "<<x<<" and y is:
            "<<y<<endl;
    }
    operator int(){    //casting operator..
        return int(x);
    }
};
int main(){
    int a;
    Sample s1(1,2);
    a=s1;              // Here, s1 is class type and a is
                        primitive type.
    cout<<"Value of a is: "<<a<<endl;
    return 0;
}
```

3. User-Defined to User-Defined Type:

```
//3. User-Defined Type to User-Defined Type..
// To implement this kind of conversion, we can use either Constructor
// or casting operator..
```

1. Using Constructor

```
#include <iostream>
using namespace std;
class Product{
    int x ,y;
public:
    void input(int a, int b){
        x=a;
        y=b;
    }
    int getx(){
        return x;
    }
    int gety(){
        return y;
    }
}
```

```

};
class Label{
    int a, b;
    public:
        Label(){}
        Label(Product p){
            a=p. getX();
            b=p. getY();
        }
        void display(){
            cout<<"Value of a is: "<<a<<" and b is:
                "<<b<<endl;
        }
};
int main(){
    Label l1;
    Product p1;
    p1.input(1,2);
    l1=p1;           //one user-defined type to another user-
                    defined type conversion.
    l1.display();
    return 0;
}

```

//2. Using Casting operator..

```

#include <iostream>
using namespace std;
class Label{
    int a ,b;
    public:
        Label(int x=0,int y=0):a(x),b(y){}
        void display(){
            cout<<"Value of a is: "<<a<<" and b is: "<<b<<endl;
        }
};
class Product{
    int x ,y;
    public:
        void input(int a ,int b){
            x=a;
            y=b;
        }
        operator Label(){           //casting operator
            return Label(x, y);
        }
};

int main(){
    Label l1;

```

```
Product p1;  
p1.input(1,2);  
l1=p1;    //one user-defined type to another user-defined type  
         conversion.  
l1.display();  
return 0;  
}
```

Khwopa College of Engineering
OOP Lab Sheet
I Year/ I Part
Faculty: Computer and Electronics
Labsheet #6

Objectives:

1. Function Overriding
2. Inheritance and its types

Tasks:

1. WAP with a class Cricketer that has data member name, age and no_of_match_played. From this class Cricketer derive two classes Bowler and Batsman. The Bowler class should have no_of_wicket as data member and Batsman class should have no_of_run and no_of_centuries as data members. Use appropriate member functions in all classes to make the program meaningful.
2. WAP with two base classes “Employee” and “Student” and a third class ‘Manager’ inherited from those two base classes. Assume that both of base classes has data members ‘name’ and ‘address’. Add suitable member function and write a meaningful program. Also illustrate the concept of function overriding.
3. Define a class named Course. Derive three classes from this class named: Mathematics, Science and Engineering. Then, derive two classes from Science named: Physics and Chemistry. Define data member and member functions as appropriate. Also illustrate the concept of function overriding and accessing overridden member from the derived class in your program.
4. WAP program to show the constructor and destructor invocation order in Multiple and Multilevel Inheritance. Also show how a parameterized base class constructor is called when derived class object is created.
5. WAP to illustrate the concept of Diamond Problem along with its solution.
6. WAP which depicts the concept of Hierarchical and Hybrid inheritance .Use appropriate classes along with its data members and member functions.

Theory:

Inheritance

Example1:

```
#include <iostream>
using namespace std;
class Base {
    private:
        int x=10;
    protected:
        int y=20;
    public:
        int z=30;
};
class Derived:public Base{
```

```

    public:
        void display(){
            cout<<"From Derived Class"<<endl;
            // cout<<x<<endl;
            //will not work //x is not accessible outside base class
            cout<<y<<endl;
            cout<<z<<endl;
        }
};
int main(){
    Derived d;
    cout<<"From main function"<<endl;
    // cout<<d.x<<endl; //will not work //x is not accessible
    outside base class
    // cout<<d.y<<endl; //will not work //y is not accessible
    outside base class except from its
    //derived class
    cout<<d.z<<endl;
    d.display();
    return 0;
}

```

Function Overriding:

Example:

```

class Parent{
    public:
        void display(){
            cout<<"Hello i m here from parent class"<<endl;
        }
};
class Child :public Parent{
    public:
        void display(){ //child class member function 'display()' overrides
            parent class 'display()'
            //member function
            cout<<"Hello i m here from child class"<<endl;
            // Parent::display(); // we can display parent's 'display()'
            as well in this way:-> First Option
        }
};
int main(){
    Child obj;
    obj.display();
    //obj.Parent::display(); //we can display parent's 'display()' as well in
    this way:->second //option
    return 0;
}

```

Function Hiding:

Example1:

```
class Parent{
public:
    void display(){
        cout<<"Hello i m here from parent class"<<endl;
    }
};
class Child:public Parent{
public:
    void display(int x){
        cout<<"Hello i m here from child class"<<endl;
    }
};
int main(){
    Child obj;
    // obj.display(); //will not work
    obj.display(1); //will work surely
    return 0;
}
```

Types Of Inheritance

1. Single Inheritance:

Example1:

```
#include <iostream>
using namespace std;
//Single Inheritance has only one base class and one derived class.
class Shape{
protected:
    int length ,width;
public:
    void input(){
        cout<<"Enter value of length: "<<endl;
        cin>>length;
        cout<<"Enter value of width: "<<endl;
        cin>>width;
    }
    void display(){
        cout<<"Length: "<<length<<" and Width: "<<width<<endl;
    }
};
class Rectangle :public Shape{
public:
    void area(){
        cout<<"Area: "<<length*width<<endl;
    }
};
int main(){
    Rectangle obj;
    obj.input();
    obj.display();
    obj.area();
}
```

```
return 0;
}
```

Example2:

```
#include <iostream>
using namespace std;
class Person{
    string Name;
    string Address;
protected:
    void input() {
        cout<<"Enter your name: "<<endl;
        cin>>Name;
        cout<<"Enter your address: "<<endl;
        cin>>Address;
    }
    void display() {
        cout<<"Name: "<<Name<<"\nAddress: "<<Address<<endl;
    }
};

class Student: public Person{
    int roll_no;
public:
    void input() {
        cout<<"Enter your roll_no. : "<<endl;
        cin>>roll_no;
        Person::input();
    }
    void display() {
        cout<<"\nRoll_No.: "<<roll_no<<endl;
        Person::display();
    }
};

int main() {
    Student s1;
    s1.input();
    s1.display();
    return 0;
}
```

2. Multi-Level Inheritance:

```
#include <iostream>
using namespace std;
class Person{
    //Base Class
    string Name;
    string Address;
protected:
    void input() {
        cout<<"Enter Your Name: "<<endl;
        cin>>Name;
        cout<<"Enter Your Address: "<<endl;
        cin>>Address;
    }
}
```

```

        void display() {
            cout<<"\nDetail: "<<endl<<"_____"<<endl<<endl;
            cout<<"Name: "<<Name<<endl<<"Address: "<<Address<<endl;
        }
};

class Employee:public Person{
    //Derived class "Employee" which is derived from
    //"Person" class
    string Dept_name;
protected:
    void input() {
        Person::input();
        cout<<"Enter Your Dept_Name: "<<endl;
        cin>>Dept_name;
    }
    void display() {
        Person::display();
        cout<<"Dept_Name: "<<Dept_name<<endl;
    }
};

class Lecturer:public Employee{
    //Derived class "Lecturer" which is derived from
    //"Employee" class
    int Lect_id;
public:
    void input() {
        Employee::input();
        cout<<"Enter Your Lect_ID: "<<endl;
        cin>>Lect_id;
    }
    void display() {
        Employee::display();
        cout<<"Lecturer Id: "<<Lect_id<<endl;
    }
};

int main(){
    Lecturer obj;
    obj.input();
    obj.display();
    return 0;
}

```

3. Multiple Inheritance:

```

#include <iostream>
using namespace std;
class Base1{
    //Base class "Base1"
protected:
    int a;
public:
    void get_a() {
        cout<<"Enter value of a: "<<endl;
        cin>>a;
    }
}

```

```

        }
};
class Base2{
    //Base class "Base2"
protected:
    int b;
public:
    void get_b(){
        cout<<"Enter the value of b: "<<endl;
        cin>>b;
    }
};
class Derived:public Base1,public Base2{
    //Derived class "Derived" which is
    //derived from "Base1" and "Base2" class
    int c;
public:
    void get_c(){
        cout<<"Enter the value of c: "<<endl;
        cin>>c;
    }
    void display(){
        cout<<"a= "<<a<<"\nb= "<<b<<"\nc= "<<c<<endl;
    }
};
int main(){
    Derived obj;
    obj.get_a();
    obj.get_b();
    obj.get_c();
    obj.display();
    return 0;
}

```

//Ambiguity in member accessing in Multiple Inheritance .

```

#include <iostream>
using namespace std;
class Base1{
    //Base class "Base1"
protected:
    int a;
public:
    void input(){
        cout<<"Enter value of a: "<<endl;
        cin>>a;
    }
};
class Base2{
    //Base class "Base2"
protected:
    int b;
public:
    void input(){

```

```

        cout<<"Enter the value of b: "<<endl;
        cin>>b;
    }
};
class Derived:public Base1,public Base2{
    //derived from "Base1" and "Base2" class
    //Derived class "Derived" which is
    int c;
public:
    void get_c(){
        cout<<"Enter the value of c: "<<endl;
        cin>>c;
    }
    void display(){
        cout<<"a= "<<a<<"\nb= "<<b<<"\nc= "<<c<<endl;
    }
};
int main(){
    Derived obj;
    obj.Base1::input();
    //will throw an error if you do this => obj.input(); //ambiguity
    //problem arises
    obj.Base2::input();
    //will throw an error if you do this => obj.input(); //ambiguity
    //problem arises
    obj.get_c();
    obj.display();
    return 0;
}

```

4. Hierarchical Inheritance:

```

#include <iostream>
using namespace std;
class Person{
    //Base Class
    string Name;
    string Address;
protected:
    void input(){
        cout<<"Enter your name: "<<endl;
        cin>>Name;
        cout<<"Enter your address: "<<endl;
        cin>>Address;
    }
    void display(){
        cout<<"Name: "<<Name<<"\nAddress: "<<Address<<endl;
    }
};
class Lecturer:public Person{
    //Derived class "Lecturer" which is derived from "Person" class
    int Lect_id;
public:
    void input(){

```

```

        cout<<"Enter Lecturer Id: "<<endl;
        cin>>Lect_id;
        Person::input ();
    }
    void display() {
        cout<<"Lecturer Id: "<<Lect_id<<endl;
        Person::display ();
    }
};
class Student:public Person{
    //Derived class "Student" which is derived from "Person" class
    int roll_no;
public:
    void input () {
        cout<<"Enter your roll_no. : "<<endl;
        cin>>roll_no;
        Person::input ();
    }
    void display() {
        cout<<"\nRoll_No.: "<<roll_no<<endl;
        Person::display ();
    }
};
int main() {
    Lecturer L1;
    cout<<"Enter Lecturer Info: "<<endl<<"_____"<<endl;
    L1.input ();
    cout<<endl<<endl;
    Student S1;
    cout<<"Enter Student Info: "<<endl<<"_____"<<endl;
    S1.input ();
    cout<<endl<<endl;
    cout<<"\nLecturer Detail: "<<endl<<"_____"<<endl;
    L1.display ();
    cout<<"\n\nStudent Detail: "<<endl<<"_____"<<endl;
    S1.display ();
    return 0;
}

```

Multipath Inheritance And Diamond Problem And Virtual Base Class:

Example#1:

Problem: Diamond Problem

```

#include <iostream>
using namespace std;
class A{
    protected:
        int a;
};
class B:public A{
    protected:
        int b;
};
class C: public A{

```

```

        protected:
            int c;
};
class D:public B,public C{
    protected:
        int d;
    public:
        void input() {
            cout<<"Enter value of a: "<<endl;
            cin>>a;
            //[Note:Without using virtual base class concept,use of 'a'
                will create problem.]
            cout<<"Enter value of b: "<<endl;
            cin>>b;
            cout<<"Enter value of c: "<<endl;
            cin>>c;
            cout<<"Enter value of d: "<<endl;
            cin>>d;
        }
        void display() {
            cout<<"Value of a: "<<a<<endl<<"Value of b: "<<b<<endl;
            // base class concept,use of 'a' will create problem.]
            //[Note:Without using //virtual
            cout<<"Value of c: "<<c<<endl<<"Value of d: "<<d<<endl;
        }
};
int main(){
    D obj;
    obj.input();
    obj.display();
    return 0;
}

```

Solution: Virtual Base Class

```

#include <iostream>
using namespace std;
class A{
    protected:
        int a;
};
class B:virtual public A{
    protected:
        int b;
};
class C:virtual public A{
    protected:
        int c;
};
class D:public B,public C{
    protected:
        int d;
    public:

```

```

        void input() {
            cout<<"Enter value of a: "<<endl;
            cin>>a;
            cout<<"Enter value of b: "<<endl;
            cin>>b;
            cout<<"Enter value of c: "<<endl;
            cin>>c;
            cout<<"Enter value of d: "<<endl;
            cin>>d;
        }
        void display() {
            cout<<"Value of a: "<<a<<endl<<"Value of b: "<<b<<endl;
            cout<<"Value of c: "<<c<<endl<<"Value of d: "<<d<<endl;
        }
    };
int main() {
    D obj;
    obj.input();
    obj.display();
    return 0;
}

```

Example#2:

Problem: Diamond Problem

```

#include <iostream>
using namespace std;
class A{
public:
    void display() {
        cout<<"From A Class"<<endl;
    }
};
class B:public A{
public:
    void show() {
        cout<<"From B Class"<<endl;
    }
};
class C:public A{
public:
    void showdata() {
        cout<<"From C Class"<<endl;
    }
};
class D:public B,public C{
public:
    void show_data() {
        cout<<"From D Class"<<endl;
    }
};

```

```

int main(){
    D obj;
    obj.display();
    //Without using virtual base class concept ,this line will create an error
    obj.show();
    obj.showdata();
    obj.show_data();
    return 0;
}

```

Solution: Virtual Base Class

```

#include <iostream>
using namespace std;
class A{
public:
    void display(){
        cout<<"From A Class"<<endl;
    }
};
class B:virtual public A{
public:
    void show(){
        cout<<"From B Class"<<endl;
    }
};
class C:virtual public A{
public:
    void showdata(){
        cout<<"From C Class"<<endl;
    }
};
class D:public B,public C{
public:
    void show_data(){
        cout<<"From D Class"<<endl;
    }
};
int main(){
    D obj;
    obj.display();
    obj.show();
    obj.showdata();
    obj.show_data();
    return 0;
}

```

[Note: Problem: Diamond Problem

Solution: Virtual Base Class]

Constructor And Destructor Invocation in Inheritance:

##Single Inheritance #Constructor_Invocation #Destructor_Invocation

```
#include <iostream>
using namespace std;
//Having default constructor and destructor in both classes: base and
//derived class
class Parent{
public:
    Parent(){
        cout<<"Hello From Parent Constructor"<<endl;
    }
    ~Parent(){
        cout<<"Parent Class Destructor"<<endl;
    }
};
class Child:public Parent{
public:
    Child():Parent(){
        cout<<"Hello From Child Constructor"<<endl;
    }
    ~Child(){
        cout<<"Child Class Destructor"<<endl;
    }
};
int main(){
    Child obj;
    return 0;
}
```

//What if base class have parameterized constructor?

//Having parameterized constructor in both parent and derived class....

```
#include <iostream>
using namespace std;
class Parent{
    int a,b;
public:
    Parent(int x,int y):a(x),b(y){
        cout<<"Hello From Parent Constructor"<<endl;
    }
    void display(){
        cout<<"a= "<<a<<endl<<"b= "<<b<<endl;
    }
    ~Parent(){
        cout<<"Parent Class Destructor"<<endl;
    }
};
```

```

class Child:public Parent{
    int c;
    public:
        Child(int m,int n,int o):Parent(n,o){
            c=m;
            cout<<"Hello From Child Constructor"<<endl;
        }
        void display(){
            cout<<"c= " <<c<<endl;
            Parent::display();
        }
        ~Child(){
            cout<<"Child Class Destructor"<<endl;
        }
};

int main(){
    Child obj(1,2,3);
    obj.display();
    return 0;
}

```

```

//Multiple Inheritance #Constructor_Invocation #Destructor_Invocation
//Having default constructor and destructor in both base classes and derived class .
#include <iostream>
using namespace std;
class Parent1{
    public:
        Parent1(){
            cout<<"Parent1 Constructor"<<endl;
        }
        ~Parent1(){
            cout<<"Parent1 Destructor"<<endl;
        }
};

class Parent2{
    public:
        Parent2(){
            cout<<"Parent2 Constructor"<<endl;
        }
        ~Parent2(){
            cout<<"Parent2 Destructor"<<endl;
        }
};

class Child:public Parent1,public Parent2{
    public:
        Child(){
            cout<<"Child Constructor"<<endl;
        }
        ~Child(){
            cout<<"Child Destructor"<<endl;
        }
}

```

```
};
int main(){
    Child obj;
    return 0;
}
```

//What if both base classes and derived class have parameterized constructor

```
#include <iostream>
using namespace std;
class Parent1{
    int a;
public:
    Parent1(int x):a(x){
        cout<<"Parent1 Constructor"<<endl;
    }
    ~Parent1(){
        cout<<"Parent1 Destructor"<<endl;
    }
    void display(){
        cout<<"a= "<<a<<endl;
    }
};
class Parent2{
    int b;
public:
    Parent2(int y):b(y){
        cout<<"Parent2 Constructor"<<endl;
    }
    ~Parent2(){
        cout<<"Parent2 Destructor"<<endl;
    }
    void display(){
        cout<<"b= "<<b<<endl;
    }
};
class Child:public Parent1,public Parent2{
    int c;
public:
    Child(int x,int y,int z):Parent1(x),Parent2(y){
        c=z;
        cout<<"Child Constructor"<<endl;
    }
    ~Child(){
        cout<<"Child Destructor"<<endl;
    }
    void display(){
        Parent1::display();
        Parent2::display();
        cout<<"c= "<<c<<endl;
    }
};
int main(){
```

```

        Child obj(1,2,3);
        obj.display();
        return 0;
}

```

// For Hierarchical Inheritance

Example#1:

```

#include <iostream>
using namespace std;
class Base{
public:
    Base() {
        cout<<"Base Class Constructor"<<endl;
    }
    ~Base() {
        cout<<"Base Class Destructor"<<endl;
    }
};
class Derived1:public Base{
public:
    Derived1() {
        cout<<"Derived1 Class Constructor"<<endl;
    }
    ~Derived1() {
        cout<<"Derived1 Class Destructor"<<endl;
    }
};
class Derived2:public Base{
public:
    Derived2() {
        cout<<"Derived2 Class Constructor"<<endl;
    }
    ~Derived2() {
        cout<<"Derived2 Class Destructor"<<endl;
    }
};
int main() {
    Derived1 obj;
    Derived2 obj1;
    return 0;
}

```

Example#2:

```

#include <iostream>
using namespace std;
class Base{
    int a;
public:
    Base(int x):a(x) {
        cout<<"Base Class Constructor"<<endl;
    }
    void display() {
        cout<<"a= "<<a<<endl;
    }
}

```

```

        ~Base() {
            cout<<"Base Class Destructor"<<endl;
        }
};
class Derived1:public Base{
    int b;
public:
    Derived1(int x,int y):Base(x),b(y){
        cout<<"Derived1 Class Constructor"<<endl;
    }
    void display(){
        Base::display();
        cout<<"b: "<<b<<endl;
    }
    ~Derived1(){
        cout<<"Derived1 Class Destructor"<<endl;
    }
};
class Derived2:public Base{
    int c;
public:
    Derived2(int x,int y):Base(x),c(y){
        cout<<"Derived2 Class Constructor"<<endl;
    }
    void display(){
        Base::display();
        cout<<"c: "<<c<<endl;
    }
    ~Derived2(){
        cout<<"Derived2 Class Destructor"<<endl;
    }
};
int main(){
    Derived1 obj(2,3);
    obj.display();
    Derived2 obj1(4,5);
    obj1.display();
    return 0;
}

```

//For Multi-Level Inheritance

```

#include <iostream>
using namespace std;
class Base{
    int a;
public:
    Base(int x):a(x){
        cout<<"Base Class Constructor Called"<<endl;
    }
    void display(){
        cout<<"a= "<<a<<endl;
    }
}

```

```

        ~Base() {
            cout<<"Base Class Destructor Called"<<endl;
        }
};
class Derived:public Base{
    int b;
public:
    Derived(int x,int y):b(x),Base(y){
        cout<<"Derived Class Constructor Called"<<endl;
    }
    void display(){
        Base::display();
        cout<<"b= "<<b<<endl;
    }
    ~Derived(){
        cout<<"Derived Class Destructor Called"<<endl;
    }
};
class Derived1:public Derived{
    int c;
public:
    Derived1(int x,int y,int z):c(x),Derived(y,z){
        cout<<"Derived1 Class Constructor Called"<<endl;
    }
    void display(){
        cout<<"c= "<<c<<endl;
        Derived::display();
    }
    ~Derived1(){
        cout<<"Derived1 Class Destructor Called"<<endl;
    }
};
int main(){
    Derived1 obj(3,2,1);
    obj.display();
    return 0;
}

```

Khwopa College of Engineering
OOP Lab Sheet
I Year/ I Part
Faculty: Computer and Electronics
Labsheet #7

Objectives:

1. Virtual Function, Pure Virtual Function and Abstract Class
2. Template (Function Template and Class Template)

Tasks:

1. WAP having polygon as an abstract class with length and height as its data members. Create derived class Rectangle and Triangle .Make Area() as pure virtual function and redefined it in derived class to calculate respective area.
2. WAP having Student as an abstract class and create derived class such as Engineering, Science and Medical. Show the use of virtual function in this program.
3. WAP to illustrate overloading of a function template with both normal and a function template.
4. WAP that will find the sum and average of elements in an array using function templates.
5. WAP with a class template to represent array and add member functions to find maximum, minimum and sort the generic array.
6. WAP using template to add two numbers. Use the function template to pass integer, char, float and double. Display the returned result.
7. WAP using class template to implement Stack (perform push () and pop() operation) for different data types like integer, character and double.